Redis 学习小结（点赞 + 扫码签到双案例实战）

一、Redis 核心认识

Redis 是一款**基于内存的高性能键值存储数据库**，读写速度极快，支持丰富的数据结构、原子操作与键过期机制，非常适合处理**高并发、高频读写、临时状态管理**类业务，可大幅减轻数据库压力，提升系统整体性能。

二、场景一：Redis 实现点赞功能

1. 业务场景

用于文章、评论、视频等内容的**点赞 / 取消点赞、点赞数统计、点赞状态查询、点赞用户列表展示**，解决高并发点赞场景下数据库性能瓶颈问题。

2. Redis 数据结构选型

- **Hash**：`like:status:{业务ID}`，存储用户点赞状态，Field 为用户 ID，Value 为 1/0，快速判断是否已点赞。
- **String**：`like:count:{业务ID}`，存储总点赞数，使用 `INCR/DECR` 实现原子增减，保证高并发下计数准确。
- **ZSet**：`like:users:{业务ID}`，存储点赞用户列表，Score 为时间戳，支持按时间倒序、分页查询。

3. 核心流程要点

- 点赞前校验状态，**防止重复点赞**。
- 采用 Redis 原子操作，无并发安全问题。
- 可设置过期时间，避免内存占用过大。
- 支持实时统计、状态查询、用户列表展示。

三、场景二：Redis 实现扫码签到功能

1. 业务场景

用于会议、课堂、活动的**二维码生成、扫码验证、一次性签到、过期失效**，实现高效、防代签、防复用的签到流程。

2. Redis 数据结构选型

- **String**：`sign:qr:{签到码}`，存储临时签到凭证，绑定活动 ID，并设置**10 分钟过期**。
- 键删除机制：签到成功后立即删除 Key，保证**一次性使用**。

3. 核心流程要点

- 生成 6 位唯一随机码，循环校验避免重复。
- 二维码内置签到链接，扫码直接访问接口。
- 校验签到码有效性（存在、未过期、未使用）。
- 失效 / 已使用二维码统一返回失败，安全可控。

四、本次学习掌握的 Redis 核心技能

1. **数据结构灵活选型**

    - Hash：适合状态集合、用户状态批量存储。
    - String：适合计数、简单键值、临时凭证。
    - ZSet：适合排序、排行榜、时间序列数据。

2. **高并发安全保障**

    - 原子操作：`INCR`、`DECR`、`HSET` 等命令天然保证并发安全。
    - 内存级读写：响应快、性能高，支撑大量请求。

3. **临时数据管理能力**

- 过期时间：自动清理临时数据，简化业务逻辑。
- 一次性凭证：用后即删，防止重复使用。

4. **工程规范与最佳实践**

- 统一 Key 前缀：`like:`、`sign:qr:`，避免键冲突。
- 状态前置校验：减少无效请求，提升效率。
- 高频操作放 Redis，数据库只做持久化。

五、学习总结

通过**点赞功能**和**扫码签到**两个真实业务场景，我掌握了 Redis 在实际项目中的**设计思路、结构选型、流程控制与安全处理**。

Redis 不仅是缓存工具，更是高并发系统中**状态管理、计数、临时凭证、排序**的核心组件。合理使用数据结构、原子性与过期策略，就能高效、安全地实现各类高频交互功能，显著提升系统性能与用户体验。